

IF2211 Teori Bahasa Formal dan Automata

Milestone 3

Semantic Analysis

Laporan Tugas Besar

Disusun untuk memenuhi tugas mata kuliah IF2224 Teori Bahasa Formal dan Otomata
pada Semester V Tahun Akademik 2025/2026



Disusun Oleh

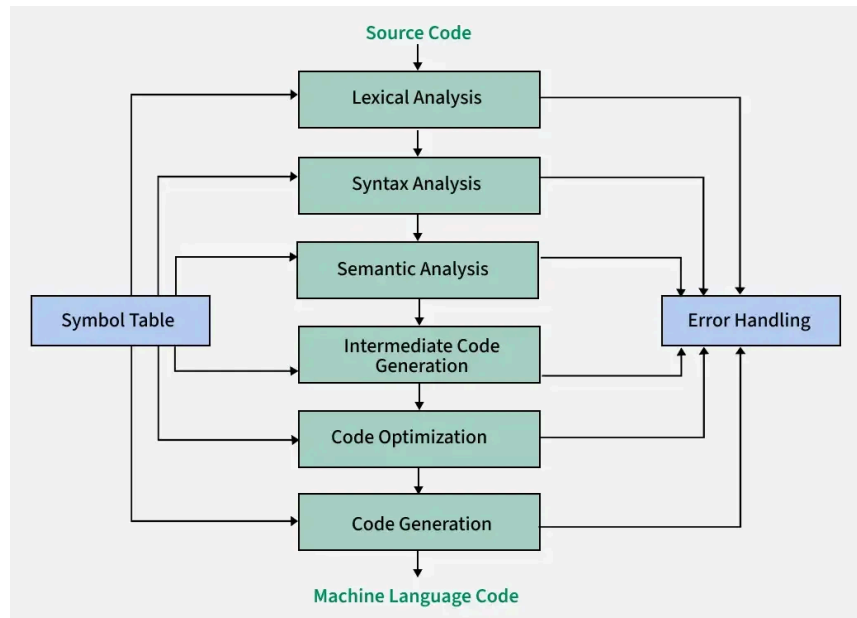
Dita Maheswari	13523125
Boye Mangaratua Ginting	13523127
Samantha Laqueenna Ginting	13523138
Lukas Raja Agripa	13523158

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1 LANDASAN TEORI.....	3
BAB 2 PERANCANGAN DAN IMPLEMENTASI.....	6
a) ObjectType.....	6
b) SymbolTable.....	6
c) Decorated AST.....	7
d) Declaration Visitor.....	9
e) Expression Visitor.....	11
f) Statement Visitor.....	14
g) Integrasi dengan Syntax Analysis.....	16
BAB 3 PENGUJIAN.....	17
a) Menguji Scope (Lokal vs Global) dan Shadowing (Valid).....	17
BAB 4 KESIMPULAN DAN SARAN.....	23
LAMPIRAN.....	25
A. Github.....	25
B. Pembagian Tugas.....	25
REFERENSI.....	26

BAB 1 LANDASAN TEORI



Gambar 1.1. Proses Compiler

Semantic analysis adalah fase dari proses kompilasi yang terjadi setelah fase analisis sintaksis (*parsing*) dan sebelum pembangkitan kode (*code generation*). Secara struktural, fase ini menerima *syntax tree* yang telah dibangun di fase *parsing*, dan menghasilkan *decorated/annotated syntax tree* yang merupakan hasil yang telah divalidasi.

Saat kode program berhasil melewati fase *parsing*, program telah dinyatakan benar secara sintaksis oleh *parser* (sesuai dengan aturan tata bahasa atau *grammar*). Namun, program tersebut belum tentu benar secara makna atau logika. Oleh karena itu, *semantic analysis* dilakukan untuk memeriksa konsistensi program terhadap definisi bahasa kode program yang digunakan. Pemeriksaan utama dalam fase *semantic analysis* meliputi:

- **Type checking:** Memastikan tipe data yang digunakan dalam operasi konsisten. Contohnya, *compiler* akan memastikan bahwa sebuah operasi penjumlahan integer dilakukan dengan menjumlahkan variabel-variabel yang bertipe integer.
- **Scope checking:** Memastikan variabel, fungsi, atau konstanta telah dideklarasikan sebelum digunakan dan diakses dalam *scope* yang benar. Contohnya, *compiler* akan mengeluarkan *semantic error* saat program mengakses sebuah variabel yang belum pernah dideklarasikan.
- **Flow control:** Memastikan aliran kontrol program valid, seperti memastikan *break* hanya dinyatakan di dalam *loop*.

Analisis semantik merupakan fase terakhir *compiler* yang melempar *error messages* pada programmer sebelum kode program diterjemahkan ke bahasa mesin.

Untuk melakukan analisis semantik, metode *context-free grammar* (CFG) perlu diperbarui menggunakan metode yang bernama *attributed grammar*. Dalam metode ini, tata bahasa dilengkapi dengan atribut dan aturan semantik untuk menghitung nilai atribut dengan tujuan untuk membawa informasi melintasi pohon.

Berdasarkan arah aliran informasi dalam *syntax tree*, atribut dibedakan menjadi dua jenis,

- **Synthesized attributes:** Nilai atribut pada sebuah *node* dihitung berdasarkan nilai atribut dari *children node*. Informasi mengalir dari bawah ke atas (*bottom-up*). Contohnya, tipe data dari ekspresi $A + B$ disintesis dari tipe A dan tipe B .
- **Inherited attributes:** Nilai atribut pada sebuah simpul dihitung berdasarkan nilai atribut dari *parent node* atau *node* yang sejajar dengannya. Informasi mengalir dari atas ke bawah (*top-down*) atau menyamping. Contohnya, informasi dari deklarasi `int a, b, c` diwariskan dari kata kunci `int` ke variabel `a`, `b`, dan `c`.

Agar nilai atribut dapat dihitung dengan benar, ketergantungan antar atribut tidak boleh membentuk siklus (*cycle*). Urutan evaluasi ini ditentukan menggunakan *dependency graph*. Jika graf tersebut bukan merupakan siklus, maka evaluasi atribut dapat dilakukan, misalnya menggunakan metode *S-Attributed* (hanya atribut sintesis) atau *L-Attributed* (sintesis dan pewarisan terbatas) yang cocok dengan *parsing top-down*.

Atribut yang dimaksud dapat berupa tipe data, nilai konstanta, atau referensi ke struktur data. Struktur data yang digunakan adalah sebuah *symbol table* yang menyimpan informasi mengenai *identifier* yang dideklarasikan dalam program. Secara teoritis, untuk menangani *nested scope* seperti pada bahasa pemrograman Pascal, *symbol table* sering diimplementasikan menggunakan struktur *stack*. Saat memasuki blok baru, tabel *scope* baru ditambahkan (*push*), dan saat keluar blok, tabel tersebut dihapus (*pop*). *Symbol table* juga dapat dibedakan menjadi tiga jenis:

- **Identifier table:** Menyimpan informasi lengkap mengenai semua identifier termasuk konstanta, variabel, prosedur, dan fungsi.
- **Block table:** Menyimpan informasi mengenai blok prosedur, fungsi, atau definisi tipe record. Tabel ini membantu mengelola ukuran variabel lokal dan parameter.
- **Array table:** Menyimpan informasi spesifik mengenai array, seperti tipe indeks, batasan indeks, dan tipe elemen.

Proses analisis semantik pada AST dilakukan menggunakan pola desain visitor. Mekanisme ini melibatkan penelusuran pohon secara rekursif. Setiap *node* pada AST memiliki fungsi visit yang dimiliki yang bertugas untuk:

- **Traversing:** Memanggil fungsi visit pada *child node*.

- **Context checking:** Mengakses *symbol table* untuk memvalidasi deklarasi dan tipe data.
- **Synthesizing:** Mengembalikan informasi ke *parent node*.
- **Annotating:** Menyimpan informasi tipe yang telah divalidasi kembali ke dalam *node* AST untuk digunakan pada tahap selanjutnya.

Dalam implementasi pola Visitor, sering kali digunakan mekanisme Dispatcher untuk menangani pemilihan metode secara dinamis. Karena Abstract Syntax Tree (AST) terdiri dari berbagai jenis node (seperti BinOp, VarDecl, Number), visitor perlu mengetahui metode spesifik mana yang harus dipanggil untuk setiap jenis node tanpa menggunakan struktur kontrol *if-else* atau *switch-case* yang panjang dan sulit dipelihara.

Mekanisme dispatcher bekerja dengan memanfaatkan prinsip Dynamic Dispatch atau Double Dispatch:

- **Delegasi Dinamis:** Sebuah metode generik bertindak sebagai gerbang utama. Metode ini memeriksa tipe kelas dari *node* yang diterima sebagai argumen.
- **Konstruksi Nama Metode:** Berdasarkan tipe *node* tersebut, dispatcher mengonstruksi nama metode target secara otomatis.
- **Eksekusi (*Invocation*):** Jika metode yang sesuai ditemukan, dispatcher akan mengeksekusinya. Jika tidak, ia dapat mengalihkan ke penanganan kesalahan atau metode *default* (*generic_visit*).

Penerapan dispatcher membuat kode program lebih modular, karena penambahan jenis *node* baru dalam tata bahasa hanya memerlukan penambahan metode *visit_Baru* yang sesuai tanpa perlu mengubah logika inti penelusuran pohon. Di akhir, validasi semantik diimplementasikan pada sistem tipe bahasa pemrograman yang diimplementasikan.

BAB 2 PERANCANGAN DAN IMPLEMENTASI

a) **ObjectType**

Class ini mendefinisikan kategori identifier beserta tipe datanya menggunakan *enumeration*. Hal ini memungkinkan SymbolTable menjaga data tiap identifier sesuai kategorinya (misal fungsi punya return type, variabel punya adr/offset).

1. **TabEntry**

Mewakili identifier dalam tabel simbol:

- id → nama identifier
- obj → object type
- type → tipe data
- ref → pointer ke tabel tipe (ARRAY/RECORD)
- lev → scope level
- adr → alamat/offset
- link → linked list untuk chaining deklarasi dalam block

2. **BTabEntry**

Merepresentasikan data block:

- last → entry terakhir di block tersebut
- lpar → jumlah parameter
- psze → ukuran parameter
- vsze → ukuran variabel lokal

3. **ATabEntry**

Representasi tipe array, yaitu tipe elemen, batas bawah dan atas, ukuran elemen, total size

b) **SymbolTable**

SymbolTable adalah komponen utama untuk menyimpan informasi semua identifier dalam program, terdiri dari tiga tabel:

- TAB (Identifier Table)
- BTAB (Block Table)
- ATAB (Array Type Table)

Struktur ini mengimplementasikan model tabular khas kompilator Pascal, dengan dukungan scope dan linked list antar deklarasi. SymbolTable juga menyimpan:

- tab → list TabEntry (identifier)
- btab → list BTabEntry (blok / scope)
- atab → list ATabEntry (tipe array)
- display → stack blok aktif
- level → depth dari scope saat ini
- built-in types, built-in constants, dan built-in procedures

Setiap deklarasi baru dimasukkan ke TAB dengan mempertimbangkan:

- tingkat scope (lev)
- linked list berdasarkan urutan deklarasi

- block size (vsze)

Berikut merupakan function-function yang diimplementasikan di dalam SymbolTable

1. Manajemen Scope

- a. enter_program
memasukkan nama program ke TAB
- b. enter_block
level scope bertambah. BTAB entry baru dibuat, display stack ditambah
- c. exit_block
scope level dikurangi, display di-pop

2. Enter Identifier (enter_identifier)

Digunakan untuk menambahkan variabel, parameter, fungsi, dan sebagainya

- menentukan lev = scope aktif
- menambah entry ke TAB
- memperbarui last dan link pada BTAB
- memperbesar ukuran block untuk variabel (vsze)

3. LookUp Identifier(lookup_identifier)

Lookup bekerja dengan cara:

- mulai dari scope terdalam → global
- mencari entry pertama (head of list) untuk setiap block
- menelusuri daftar linked list
- jika ditemukan → kembalikan indeks
- jika identifier adalah built-in → masukkan ke TAB (lazy loading)
- jika tidak ada → return None (undefined identifier error)

Mekanisme ini memastikan:

- shadowing antar scope berjalan benar
- reserved words tidak ikut dicari
- built-in constant & type dikenali tanpa disimpan di TAB

c) Decorated AST

Bagian ini membahas struktur data *Decorated Abstract Syntax Tree* (Decorated AST), yang merupakan hasil keluaran utama dari proses *Semantic Analysis*. Berbeda dengan AST sebelumnya yang hanya merepresentasikan struktur sintaksis dari *Parser*, Decorated AST diperkaya dengan informasi semantik yang krusial untuk tahap pembuatan kode (*Code Generation*).

1. Desain Kelas Dasar (DecoratedASTNode)

Seluruh *node* dalam pohon semantik diturunkan dari kelas dasar *DecoratedASTNode* yang mewarisi *ASTNode* dari *Parser*. Kelas ini menambahkan atribut-atribut "dekorasi" yang diisi selama proses *visiting* oleh *Semantic Analyzer*:

1. `data_type`: Menyimpan tipe data hasil evaluasi node (misalnya INTEGER, BOOLEAN, atau VOID). Ini digunakan untuk pengecekan tipe (*type checking*).
2. `tab_index`: Menyimpan indeks referensi ke *Symbol Table* (tab). Ini menghubungkan penggunaan variabel/prosedur di AST dengan informasi deklarasinya.
3. `block_index`: Menyimpan indeks referensi ke *Block Table* (btab), digunakan pada node deklarasi prosedur/fungsi untuk menandai dimulainya *scope* baru.
4. `scope_level`: Menyimpan tingkat kedalaman *scope* (lev) di mana node tersebut berada.

Metode `annotate()` disediakan untuk mempermudah pengisian atribut-atribut tersebut secara fleksibel saat node dikunjungi.

2. Struktur Node Spesifik

Implementasi Decorated AST membagi node ke dalam beberapa kategori berdasarkan fungsinya dalam program Pascal-S:

1. Node Struktural dan Deklarasi
 - `ProgramASTNode`: Node akar yang menampung daftar deklarasi global dan blok program utama.
 - `VarDeclASTNode`: Merepresentasikan deklarasi variabel yang telah dianotasi dengan tipe data dan posisinya di tabel simbol.
 - `ProcedureDeclASTNode` & `FunctionDeclASTNode`: Merepresentasikan sub-program. Node ini menyimpan daftar parameter formal, deklarasi lokal, dan blok kode prosedur tersebut.
2. Node Pernyataan (*Statement*)

Node ini merepresentasikan logika eksekusi dan umumnya memiliki tipe data VOID.

 - `AssignASTNode`: Merepresentasikan operasi penugasan (`:=`). Node ini menyimpan target (variabel tujuan) dan value (ekspresi sumber).
 - `IfASTNode`, `WhileASTNode`, `RepeatASTNode`: Merepresentasikan struktur kontrol. Node ini menyimpan referensi ke node kondisi (*condition*) dan blok pernyataan yang dieksekusi.
 - `ForASTNode`: Menyimpan informasi variabel iterasi, nilai awal (*start*), nilai akhir (*end*), dan arah iterasi (*to/down to*).
3. Node Ekspresi dan Faktor

Node ini adalah node daun atau cabang yang selalu menghasilkan nilai dan tipe data (`data_type` tidak boleh None).

 - `Literal Node`: `NumberASTNode` (Integer/Real), `StringASTNode`, `BooleanASTNode`, dan `CharASTNode`.
 - `VarASTNode`: Representasi penggunaan variabel dalam ekspresi.

- BinOpASTNode & UnaryOpASTNode: Representasi operasi aritmatika, logika, atau relasional. Node ini menyimpan operator dan operand (kiri/kanan).
4. Akses Memori Kompleks
 - ArrayAccessASTNode: Menangani akses elemen array dengan menyimpan referensi array dan ekspresi indeksinya.
 - RecordAccessASTNode: Menangani akses *field* record (record.field).
 - ProcCallASTNode & FuncCallASTNode: Menangani pemanggilan sub-program beserta daftar argumen aktualnya.

3. Visualisasi Pohon

Untuk keperluan menampilkan hasil dari decorated AST, setiap kelas node mengimplementasikan metode `to_string()` yang menghasilkan representasi string hierarkis.

Visualisasi ini menggunakan karakter *box-drawing* Unicode (|, └─, └─) untuk menggambarkan struktur pohon dengan jelas. Selain struktur, visualisasi ini juga menampilkan anotasi semantik di samping setiap node, seperti:

- type:integer
- tab_index:30
- lev:1

Fitur ini sangat membantu dalam memverifikasi apakah *Semantic Analyzer* telah berhasil menempelkan tipe data dan referensi tabel simbol dengan benar.

d) Declaration Visitor

Bagian ini membahas perancangan dan implementasi DeclarationVisitor, sebuah modul dalam *Semantic Analyzer* yang bertugas memvalidasi semantik pada blok deklarasi. Tujuan utama dari visitor ini adalah memastikan bahwa setiap deklarasi program mematuhi aturan tipe data dan logika deklarasi bahasa Pascal-S.

1. Desain dan Arsitektur

DeclarationVisitor diimplementasikan sebagai kelas turunan dari SemanticAnalyzerBase. Modul ini bekerja dengan menelusuri node dalam *Abstract Syntax Tree* (AST). Karena struktur AST menyimpan daftar deklarasi dalam DeclarationPartNode, visitor ini menerapkan pola desain Dispatcher.

2. Logika Validasi Semantik

Berikut adalah logika utama yang diimplementasikan dari DeclarationVisitor:

1) Validasi Variable Declaration

Pada deklarasi variabel (`variabel := type`), visitor melakukan serangkaian pemeriksaan ketat:

- Keberadaan Identifier

Memeriksa apakah identifier di sisi kiri sudah dideklarasikan dalam Symbol Table di scope saat ini. Jika sudah ada, kesalahan Redclaration Error akan dibangkitkan.

- Validasi Tipe

Memastikan bahwa tipe yang dideklarasikan adalah tipe yang valid dan dikenal sistem (seperti INTEGER, REAL, BOOLEAN, CHAR, STRING, atau tipe array).

- Registrasi ke Symbol Table

Setiap variabel yang valid akan didaftarkan ke dalam Symbol Table dengan informasi: nama identifier, ObjectType.VARIABLE, tipe data, dan reference/indirection level (untuk array).

- Pembuatan AST Node

Setiap deklarasi variabel akan menghasilkan VarDeclASTNode yang dianotasi dengan tipe data, tab_index (posisi di Symbol Table), dan scope_level.

2) Validasi Procedure Declaration

Untuk deklarasi prosedur, visitor memvalidasi struktur dan konsistensi scope:

- Registrasi Pra-Blok: Nama prosedur didaftarkan ke *Symbol Table* pada *scope* aktif sebelum *enter_block()* dipanggil. Hal ini memastikan prosedur sudah dikenali saat memproses isi bloknnya sendiri (mendukung rekursi).
- Pembuatan Block Baru: Prosedur menciptakan *scope* baru (Level +1) dengan memanggil *enter_block()*, mengisolasi parameter dan variabel lokal dari variabel global.
- Validasi Parameter & Body: Parameter formal diproses dan dimasukkan ke tabel simbol lokal, diikuti dengan pemrosesan deklarasi lokal dan *compound statement* menggunakan *StatementVisitor*.
- Pembersihan Scope (Exit Block): Di akhir kunjungan, *exit_block()* dipanggil secara eksplisit untuk menurunkan level *scope*. Ini mencegah kebocoran *scope* di mana variabel lokal prosedur secara keliru terdeteksi di blok program utama.

3) Validasi Function Declaration

Deklarasi fungsi memiliki validasi yang mirip dengan prosedur, dengan tambahan:

- Pemisahan Tahap (Scanning vs Processing): Visitor melakukan pemindaian awal untuk mengekstrak nama fungsi dan tipe kembalian (*return type*) terlebih dahulu.
- Registrasi Simbol: Fungsi didaftarkan ke *Symbol Table* sebelum masuk ke blok barunya. Ini memungkinkan mekanisme *assignment* ke nama fungsi (sebagai *return value*) valid dilakukan di dalam tubuh fungsi tersebut.

- Manajemen Blok: Sama seperti prosedur, fungsi membuat *scope* baru untuk parameter dan variabel lokalnya.
- 4) Validasi Constant dan Type Declaration
- Untuk deklarasi konstanta (*ConstDeclarationNode*) dan tipe (*TypeDeclarationNode*):
- Saat ini ditandai sebagai TODO untuk implementasi di masa depan.
 - Struktur sudah disiapkan untuk mengembalikan list of *ConstDeclASTNode* atau *TypeDeclASTNode*.
3. Integrasi Modular
- DeclarationVisitor* tidak bekerja sendiri, melainkan berintegrasi dengan komponen lain:
- *SymbolTable*
Digunakan untuk mencatat setiap identifier yang dideklarasikan, memeriksa redeclaration, dan mengelola *scope hierarchy*.
 - *StatementVisitor*
Ketika memproses prosedur atau fungsi, *DeclarationVisitor* memanggil *StatementVisitor* untuk memvalidasi compound statement di dalamnya.
 - *ExpressionVisitor*
Ketika ada constant declaration dengan ekspresi, *ExpressionVisitor* akan dipanggil untuk mengevaluasi tipe ekspresi tersebut.
 - SemanticError Handling
Semua error seperti *RedeclarationError* atau *UndeclaredVariable* dibangkitkan melalui custom exception classes untuk error reporting yang jelas.

e) Expression Visitor

Bagian ini membahas perancangan dan implementasi *ExpressionVisitor*, sebuah modul dalam *Semantic Analyzer* yang bertugas melakukan pengecekan semantik pada ekspresi dalam bahasa Pascal-S. Visitor ini tidak hanya mengevaluasi struktur ekspresi (operator–operand), tetapi juga memastikan kesesuaian tipe, komposisi, dan kompatibilitas operasi.

1. Desain dan Arsitektur

ExpressionVisitor diimplementasikan sebagai subclass dari *SemanticAnalyzerBase*. Ia beroperasi dengan menelusuri node ekspresi pada Abstract Syntax Tree (AST), terutama node:

- *ExpressionNode*
- *SimpleExpressionNode*
- *TermNode*
- *FactorNode*

Struktur grammar Pascal-S untuk ekspresi sangat hirarkis: faktor → term → ekspresi sederhana → ekspresi relasional. Oleh karena itu, desain modul menerapkan pola visitor bersarang (nested dispatch), di mana setiap level menangani tipe operator dan aturan semantik yang berbeda.

Visitor ini selalu menghasilkan DecoratedASTNode, yaitu node AST yang sudah memiliki anotasi tipe (`data_type`), indeks simbol (`tab_index`), atau informasi lain hasil analisis semantik.

2. Logika Validasi Semantik

a. Analisis Ekspresi Relasional (`visit_expression`)

Singkatnya adalah `simple-expression (relational-op simple-expression)?`

Visitor memecahnya menjadi operand kiri (`left`) dan operand kanan (`right`).

Bila operator relasional ditemukan:

- Visitor memeriksa kesesuaian tipe kedua operand.
- Memperbolehkan operasi numerik campuran (`INTEGER` vs `REAL`).
- Menghasilkan `BinOpASTNode` yang selalu bertipe `BOOLEAN`.

Tujuan dari analisis ini adalah memastikan ekspresi seperti ``a <= b * 2.5`` valid dan menghasilkan tipe logika. Jika tidak kompatibel, maka akan return `TypeMismatchError`

b. Analisis Ekspresi Aritmetika & Logika (`visit_simple_expression`)

Analisis ini ini menangani:

- Unary operator (`+x`, `-x`)
- Operator aritmatika penjumlahan / pengurangan
- Operator logika OR (`atau`)

Visitor membangun pohon AST secara kiri-ke-kanan:

- Unary diproses terlebih dahulu $\rightarrow$ dibuat `UnaryOpASTNode`
- Operasi biner (`+`, `-`, `atau`) $\rightarrow$ dibangun melalui `_create_binary_op`

Konsistensi tipe:

- Operator `+`, `-` hanya untuk numerik
- `"atau"` hanya untuk boolean
- *String concatenation* khusus: `"abc" + "def"`

c. Analisis Term Multiplikatif (`visit_term`)

Menangani operator:

- `*`, `/`
- `bagi`
- `mod`
- Logika `dan`

Aturan utama:

- Semua operand operator di atas harus bertipe numerik kecuali `dan`
- Pembagian `/` $\rightarrow$ hasil `REAL`

- bagi dan mod → hasil INTEGER
- Logika dan → hasil BOOLEAN

Pembangunan AST selalu dilakukan berurutan:

`factor1 op factor2 → BinOp → BinOp op factor3 → ...`

d. Analisis Faktor (`visit_factor`)

Faktor merupakan unit atom ekspresi:

- Literal (NUMBER, STRING_LITERAL, CHAR_LITERAL)
- IDENTIFIER
- Pemanggilan fungsi (CallStatementNode)
- Kurung (expression)
- Unary logical `tidak`

Karakteristik penting:

- Literal langsung menghasilkan AST node bertipe final (NumberASTNode, StringASTNode, dll).
- Identifier tidak langsung diterjemahkan → didelegasikan ke ArrayAccessVisitor (karena bisa variable, array, record).
- Function call didelegasikan ke ProcFuncVisitor → mengembalikan return type fungsi.
- Operator `tidak` dievaluasi tipe operandnya harus BOOLEAN.

Semua error operator salah tipe menggunakan custom exception seperti *InvalidOperationError*.

3. Mekanisme Type Checking

a. `_create_binary_op`

Metode ini adalah inti validasi:

- Infer tipe operand jika belum dianotasi.
- Cek kompatibilitas operator terhadap operand.
- Hasilkan BinOpASTNode yang sudah diberi anotasi tipe.

b. `_check_type_compatibility`

Metode ini berperan sebagai *type rules engine*:

- Aritmetika: *numeric only*
- Boolean: operand boolean wajib
- Relasional: tipe sama atau *numeric-mixed*
- String concat: hanya STRING + STRING

4. Inferensi Tipe AST Node (`_infer_expression_type`)

Metode ini mengantisipasi kode yang terkadang belum teranotasi, misalnya AST yang dibangun oleh visitor lain. Visitor ini memprediksi tipe berdasarkan jenis literal, *symbol table entry*, node operator(bin/unary), dan result

record/array/fungsi. Inferensi ini memastikan sistem tetap konsisten dalam kasus *nested expression*.

5. Integrasi Modular

ExpressionVisitor tidak bekerja sendiri dan berkolaborasi dengan komponen lain:

a. SymbolTable

- *Lookup* tipe variabel
- Ambil indeks simbol fungsi
- Mendukung inferensi tipe *VarASTNode*

b. DeclarationVisitor

- Menentukan tipe deklarasi *var/func* sebelumnya
- *ExpressionVisitor* mengonsumsi informasi ini

c. ProcFuncVisitor

- Untuk function call
- Menjamin parameter sesuai tipe formal
- Menghasilkan AST fungsi dengan return type

d. ArrayAccessVisitor

- Saat ekspresi memuat array indexing
- Menentukan tipe elemen array
- Mengecek batas dimensi

6. SemanticError Handling

ExpressionVisitor menghasilkan error dengan konteks lengkap:

- *InvalidOperationError* (operator tidak valid untuk tipe)
- *TypeMismatchError* (kompatibilitas tipe gagal)
- *SemanticError* umum (node tidak valid)
- *Redeclared/Undeclared* direlay dari level simbol

f) Statement Visitor

Bagian ini membahas perancangan dan implementasi *StatementVisitor*, sebuah modul dalam *Semantic Analyzer* yang bertugas memvalidasi semantik pada blok kode eksekusi (*executable statements*). Tujuan utama dari visitor ini adalah memastikan bahwa setiap instruksi program mematuhi aturan tipe data (*type system*) dan logika alur kontrol Pascal-S sebelum kode dieksekusi atau diterjemahkan.

1. Desain dan Arsitektur

StatementVisitor diimplementasikan sebagai kelas turunan dari *SemanticAnalyzerBase*. Modul ini bekerja dengan menelusuri *node* pernyataan dalam *Abstract Syntax Tree* (AST) yang dihasilkan oleh *parser*. Karena struktur

AST menyimpan daftar pernyataan dalam bentuk generik (*StatementListNode*), visitor ini menerapkan pola desain Dispatcher.

Metode *visit_statement* bertindak sebagai *dispatcher* pusat yang memeriksa tipe *node* AST (menggunakan *isinstance*) dan mengarahkan eksekusi ke metode validasi yang spesifik, seperti *visit_assignment_statement*, *visit_if_statement*, atau *visit_while_statement*.

2. Logika Validasi Semantik

Berikut adalah logika utama yang diimplementasikan dari *StatementVisitor*:

1) Validasi Assignment

Pada operasi penugasan (*variable := expression*), visitor melakukan serangkaian pemeriksaan ketat:

- Keberadaan Variabel: Memeriksa apakah *identifier* di sisi kiri (*Left-Hand Side*) sudah dideklarasikan dalam *Symbol Table*. Jika tidak, kesalahan *Undeclared Variable* akan dibangkitkan.
- Validasi Objek: Memastikan bahwa target penugasan adalah objek yang valid untuk diubah nilainya (seperti Variabel atau *Return Value* fungsi), bukan Konstanta atau Nama Tipe.
- Kompatibilitas Tipe: Memanggil *ExpressionVisitor* untuk menyimpulkan tipe data ekspresi di sisi kanan. Tipe data kedua sisi harus sama, dengan pengecualian *implicit casting* di mana tipe *INTEGER* diizinkan untuk dimasukkan ke variabel bertipe *REAL*.

2) Validasi Control Flow

Untuk pernyataan kondisional (*IF*, *WHILE*, *REPEAT*), visitor memvalidasi ekspresi kondisi yang mengendalikan alur program:

- Visitor mengevaluasi ekspresi kondisi dan memastikan tipe data hasil evaluasinya adalah *BOOLEAN*.
- Jika ekspresi menghasilkan tipe lain (misalnya aritmatika *INTEGER*), visitor akan membangkitkan *Type Mismatch Error*.

3) Validasi For Loop

Pernyataan *FOR* divalidasi sesuai standar Pascal yang ketat:

- Variabel iterasi (*loop variable*) harus bertipe ordinal (*INTEGER*) dan harus sudah dideklarasikan.
- Ekspresi batas awal (*start*) dan batas akhir (*end*) harus dievaluasi menjadi tipe *INTEGER*.

4) Integrasi Modular

StatementVisitor tidak bekerja sendiri, melainkan berintegrasi dengan komponen lain:

- *ExpressionVisitor*: Digunakan untuk menganalisis tipe data dari setiap ekspresi matematika atau logika yang muncul dalam pernyataan.
- *ProcFuncVisitor*: Validasi pemanggilan prosedur (*Procedure Call*) didelegasikan ke visitor ini untuk memeriksa kesesuaian jumlah dan tipe parameter aktual terhadap parameter formal.

g) Integrasi dengan Syntax Analysis

Syntax Analysis yang sudah dibuat pada milestone 2 sebelumnya, dan Semantic Analysis ini diintegrasikan melalui Abstract Syntax Tree. Tetap dari entry-point menuju ParseTree yang telah direturn dari Services.processFile (sesuai pada milestone 2). Nah sekarang di pipeline Semantic Integration di processFileWithSemantics, dimulai dari pendapatan token dari milestone 1 (Lexical Analysis), dilanjutkan Syntax Analysis yang mengenkapsulasi performSyntaxAnalysis dari token yang didapatkan dan return parseTree, dan sekarang untuk milestone 3, dilanjutkan performSemanticAnalysis dari parseTree yang didapatkan yang disimpan dalam decoratedAST. Mungkin sempat bertanya, *nah parse tree kemarin kan ada tree yang terbentuk, ataupun kenapa tidak output langsung dari txt yang didapatkan kemarin*. Ya tentu saja, supaya tidak terganggu oleh view antar muka tree yang dibuat. Nah, untuk perihal performSemanticAnalysis nya bagaimana, bisa dilihat dari point sebelumnya.

BAB 3 PENGUJIAN

- a) Menguji Scope (Lokal vs Global) dan Shadowing (Valid).
Semantic Analyzer harus tahu bahwa x di dalam prosedur cekLokal adalah boolean, sedangkan x di program utama adalah integer.

Program11.pas

Input	<pre>test > milestone-3 > input > Program11.pas 1 program TestScope; 2 variabel 3 x, y: integer; 4 5 prosedur cekLokal; 6 variabel 7 x: boolean; { Shadowing: x lokal menutupi x global } 8 mulai 9 x := true; 10 jika x maka 11 writeln('Ini x lokal (boolean)'); 12 selesai; 13 14 mulai 15 x := 10; { Ini akses x global } 16 cekLokal(); 17 y := x + 5; 18 writeln(y); 19 selesai.</pre>
Output	<pre>test > milestone-3 > output > program11_ast.txt 1 === Decorated AST === 2 ProgramNode(name: 'TestScope') 3 Declarations 4 VarDecl('x') -> tab_index:30, type:integer, lev:0 5 VarDecl('y') -> tab_index:31, type:integer, lev:0 6 ProcedureDecl -> 'cekLokal', type:void, tab_index:32, block_index:1, lev:1 7 Block -> block_index:0, lev:0 8 Assign('x' := 10) -> type:void 9 target 'x' -> tab_index:30, type:integer 10 value 10 -> type:integer 11 cekLokal(args: 0) -> tab_index:32 12 Assign('y' := x+5) -> type:void 13 target 'y' -> tab_index:31, type:integer 14 value BinOp '+' -> type:integer 15 'x' -> tab_index:30, type:integer 16 5 -> type:integer 17 writeln(args: 1) -> tab_index:34 18 'y' -> tab_index:31, type:integer</pre>

```

test > milestone-3 > output >  program11_syntab.txt
1
2 =====
3 | SYMBOL TABLE (TAB) - Identifier Table |
4 =====
5 | Idx | Identifier | Object | Type | Ref | Nrm | Lev | Adr | Link |
6 |-----|
7 | 29 | TestScope | program | 0 | 0 | 1 | 0 | 0 | 0 |
8 | 30 | x | variable | 1 | 0 | 1 | 0 | 0 | 0 |
9 | 31 | y | variable | 1 | 0 | 1 | 0 | 0 | 30 |
10 | 32 | cekLokal | procedure | 0 | 1 | 1 | 0 | 0 | 31 |
11 | 33 | x | variable | 3 | 0 | 1 | 1 | 0 | 0 |
12 | 34 | writeln (predefined) | procedure | 0 | 0 | 1 | 0 | 0 | 0 |
13 =====
14 Note: Reserved words (idx 0-28), built-in types, dan konstanta built-in tidak ditampilkan.
15
16 =====
17 | BLOCK TABLE (BTAB) |
18 =====
19 | Idx | Last | Lpar | Psze | Vsze |
20 |-----|
21 | 0 | 32 | 0 | 0 | 2 |
22 | 1 | 33 | 0 | 0 | 1 |
23 |-----|
24 =====

```

- b) Menguji Type Checking dan Implicit Casting (Valid).
Menguji apakah integer bisa dimasukkan ke real dan menguji operasi aritmatika campuran.

Program12.pas

Input	<pre> test > milestone-3 > input >  Program12.pas 1 program TestTipeData; 2 variabel 3 angka: integer; 4 pecahan: real; 5 hasil: real; 6 7 mulai 8 angka := 5; 9 pecahan := 2.5; 10 11 { Valid: Assign Integer ke Real (Implicit Casting) } 12 hasil := angka; 13 14 { Valid: Operasi Aritmatika Campuran (Int + Real = Real) } 15 hasil := angka + pecahan; 16 17 writeln('Hasil = ', hasil); 18 selesai. </pre>
-------	--

Output

```
test > milestone-3 > output >  program12_ast.txt

1  === Decorated AST ===
2  ProgramNode(name: 'TestTipeData')
3  |  └─ Declarations
4  |  |  └─ VarDecl('angka') -> tab_index:30, type:integer, lev:0
5  |  |  |  └─ VarDecl('pecahan') -> tab_index:31, type:real, lev:0
6  |  |  |  |  └─ VarDecl('hasil') -> tab_index:32, type:real, lev:0
7  |  |  └─ Block -> block_index:0, lev:0
8  |  |  |  └─ Assign('angka' := 5) -> type:void
9  |  |  |  |  └─ target 'angka' -> tab_index:30, type:integer
10 |  |  |  |  |  └─ value 5 -> type:integer
11 |  |  |  |  └─ Assign('pecahan' := 2.5) -> type:void
12 |  |  |  |  |  └─ target 'pecahan' -> tab_index:31, type:real
13 |  |  |  |  |  |  └─ value 2.5 -> type:real
14 |  |  |  |  └─ Assign('hasil' := angka) -> type:void
15 |  |  |  |  |  └─ target 'hasil' -> tab_index:32, type:real
16 |  |  |  |  |  |  └─ value 'angka' -> tab_index:30, type:integer
17 |  |  |  |  └─ Assign('hasil' := angka+pecahan) -> type:void
18 |  |  |  |  |  └─ target 'hasil' -> tab_index:32, type:real
19 |  |  |  |  |  |  └─ value BinOp '+' -> type:real
20 |  |  |  |  |  |  |  └─ 'angka' -> tab_index:30, type:integer
21 |  |  |  |  |  |  |  |  └─ 'pecahan' -> tab_index:31, type:real
22 |  |  |  |  └─ writeln(args: 2) -> tab_index:33
23 |  |  |  |  |  └─ "'Hasil = '" -> type:string
24 |  |  |  |  |  |  └─ 'hasil' -> tab_index:32, type:real

test > milestone-3 > output >  program12_syntab.txt

1
2  =====
3  |  SYMBOL TABLE (TAB) - Identifier Table
4  |  =====
5  |  |  Idx | Identifier | Object | Type | Ref | Nrm | Lev | Adr | Link |
6  |  |-----|
7  |  | 29 | TestTipeData | program | 0 | 0 | 1 | 0 | 0 | 0 |
8  |  | 30 | angka | variable | 1 | 0 | 1 | 0 | 0 | 0 |
9  |  | 31 | pecahan | variable | 2 | 0 | 1 | 0 | 0 | 30 |
10 |  | 32 | hasil | variable | 2 | 0 | 1 | 0 | 0 | 31 |
11 |  | 33 | writeln (predefined) | procedure | 0 | 0 | 1 | 0 | 0 | 0 |
12 |  |-----|
13 |  Note: Reserved words (idx 0-28), built-in types, dan konstanta built-in tidak ditampilkan.
14 |  =====
15 |  |  BLOCK TABLE (BTAB)
16 |  |  =====
17 |  |  |  Idx | Last | Lpar | Psze | Vsze |
18 |  |  |-----|
19 |  |  | 0 | 32 | 0 | 0 | 3 |
20 |  |  |-----|
21 |  |-----|
22 |  =====
```

c) Menguji Deklarasi Fungsi dan Return Value (Valid).

Memastikan nama fungsi (faktorial) sudah masuk ke symbol table sehingga bisa dipanggil di dalam dirinya sendiri (rekursi) dan memastikan nilai dikembalikan dengan cara nama_fungsi := nilai.

Program13.pas

Input

```
test > milestone-3 > input > Program13.pas

1  program TestFungsiRekursif;
2  variabel
3  |   hasil: integer;
4
5  fungsi faktorial(n: integer) : integer;
6  mulai
7  |   jika n <= 1 maka
8  |   |   faktorial := 1
9  |   selain-itu
10 |   |   faktorial := n * faktorial(n - 1);
11 selesai;
12
13 mulai
14 |   hasil := faktorial(5);
15 |   writeln('Faktorial 5 = ', hasil);
16 selesai.
```

Output

```
test > milestone-3 > output > program13_ast.txt
1  === Decorated AST ===
2  ProgramNode(name: 'TestFungsiRekursif')
3  |  Declarations
4  |  |  VarDecl('hasil') -> tab_index:30, type:integer, lev:0
5  |  |  FunctionDecl -> 'faktorial', type:integer, tab_index:31, block_index:1, lev:1
6  |  |  Block -> block_index:0, lev:0
7  |  |  |  Assign('hasil' := ...) -> type:void
8  |  |  |  |  target 'hasil' -> tab_index:30, type:integer
9  |  |  |  |  |  value faktorial(args: 1) -> type:integer
10 |  |  |  5 -> type:integer
11 |  |  |  |  writeln(args: 2) -> tab_index:33
12 |  |  |  |  |  "'Faktorial 5 = '" -> type:string
13 |  |  |  |  |  'hasil' -> tab_index:30, type:integer
```

```
test > milestone-3 > output > program13_symtab.txt

1
2  =====
3  | SYMBOL TABLE (TAB) - Identifier Table |
4  =====
5  | Idx | Identifier | Object | Type | Ref | Nrm | Lev | Adr | Link |
6  |-----|
7  | 29 | TestFungsiRekursif | program | 0 | 0 | 1 | 0 | 0 | 0 |
8  | 30 | hasil | variable | 1 | 0 | 1 | 0 | 0 | 0 |
9  | 31 | faktorial | function | 1 | 1 | 1 | 0 | 0 | 30 |
10 | 32 | n | parameter | 1 | 0 | 1 | 1 | 0 | 0 |
11 | 33 | writeln (predefined) | procedure | 0 | 0 | 1 | 0 | 0 | 0 |
12 =====
13 Note: Reserved words (idx 0-28), built-in types, dan konstanta built-in tidak ditampilkan.
14
15 =====
16 | BLOCK TABLE (BTAB) |
17 =====
18 | Idx | Last | Lpar | Psze | Vsze |
19 |-----|
20 | 0 | 31 | 0 | 0 | 1 |
21 | 1 | 32 | 32 | 0 | 0 |
22 |-----|
23 =====
```

d) Menguji Type Mismatch pada *Assignment* dan *Control Flow* (Error)

Mencoba memasukkan boolean ke integer (harus error) dan mencoba menggunakan integer sebagai kondisi jika (harus error, karena kondisi wajib boolean).

Program14.pas

Input	<pre>test > milestone-3 > input >  Program14.pas 1 program ErrorTipeData; 2 variabel 3 angka: integer; 4 status: boolean; 5 6 mulai 7 status := true; 8 9 { ERROR 1: Tidak bisa assign boolean ke integer } 10 angka := status; 11 12 { ERROR 2: Kondisi JIKA harus boolean, tapi ini integer } 13 jika angka maka 14 writeln('Ini salah'); 15 selesai.</pre>
Output	<pre>PS C:\Users\LENOVO\Documents\Github\PSC-Tubes-IF2224\src> python run.py -s ../test/milestone-3/input/program14.pas Mode: Processing Pascal file with semantic analysis: ../test/milestone-3/input/program14.pas Semantic error: Type mismatch in assignment to 'angka': expected INTEGER, got BOOLEAN ERROR: Semantic error: Semantic error: Type mismatch in assignment to 'angka': expected INTEGER, got BOOLEAN</pre>

e) Menguji Undeclared Identifier dan Konstanta.

Mencoba menggunakan variabel z yang belum dideklarasikan dan mencoba mengubah nilai konstanta (jika Anda mengimplementasikan fitur konstanta).

Program15.pas

Input	<pre> test > milestone-3 > input > Program15.pas 1 program ErrorVariabel; 2 konstanta 3 PI = 3.14; 4 variabel 5 r: real; 6 7 mulai 8 r := 10.0; 9 10 { ERROR 1: Variabel 'luas' tidak dideklarasikan } 11 luas := PI * r * r; 12 13 { ERROR 2: Konstanta tidak boleh diubah nilainya } 14 PI := 3.14159; 15 selesai. </pre>
Output	<pre> PS C:\Users\LENOVO\Documents\GitHub\PSC-Tubes-IF2224\src> python run.py -s ../test/milestone-3/input/program15.pas Mode: Processing Pascal file with semantic analysis: ../test/milestone-3/input/program15.pas Semantic error: Undeclared variable 'luas' ERROR: Semantic error: Semantic error: Undeclared variable 'luas' </pre>

BAB 4 KESIMPULAN DAN SARAN

a) Kesimpulan

Pengerjaan Milestone 3 ini telah berhasil mengimplementasikan tahap Semantic Analysis (analisis semantik) untuk compiler Pascal-S. Proses analisis ini dikerjakan dengan melakukan traversal pada Abstract Syntax Tree (AST) yang merupakan hasil dari parser pada Milestone 2, menggunakan pola desain Visitor Pattern.

Fungsi utama semantic analyzer yang dibuat adalah untuk memverifikasi apakah program sumber mematuhi aturan semantik PASCAL-S, khususnya dalam hal type system (sistem tipe data) dan scope management (pengelolaan ruang lingkup). Semantic analyzer memvalidasi kesesuaian tipe data pada operasi aritmatika dan logika, memeriksa deklarasi dan penggunaan variabel, memastikan kompatibilitas parameter pada pemanggilan prosedur/fungsi, serta mengelola scope hierarchy melalui Symbol Table. Proses ini menghasilkan Decorated AST yang diperkaya dengan informasi tipe dan Symbol Table yang mencatat seluruh identifier beserta atributnya.

Implementasi semantic analyzer dibagi secara modular menjadi beberapa visitor spesifik (DeclarationVisitor, StatementVisitor, ExpressionVisitor, dan ProcFuncVisitor) untuk mengelola kompleksitas validasi semantik dan memfasilitasi pengerjaan tim. Selain itu, analyzer juga dilengkapi dengan mekanisme deteksi error semantik yang komprehensif (seperti UndeclaredVariable, RedeclarationError, TypeMismatchError, dan ParameterMismatchError) yang memberikan pesan informatif untuk membantu programmer mengidentifikasi kesalahan logika program.

b) Saran

Adapun saran yang dapat dipertimbangkan untuk pengembangan di lain waktu ialah:

1. **Peningkatan *Type Checking*:** Meskipun validasi tipe dasar telah diimplementasikan, *semantic analyzer* dapat ditingkatkan dengan dukungan untuk *type coercion* (konversi tipe implisit) yang lebih canggih, misalnya konversi otomatis dari INTEGER ke REAL pada operasi campuran, serta validasi yang lebih ketat untuk tipe array dan record.
2. **Optimalisasi *Symbol Table*:** *Symbol Table* saat ini menggunakan struktur linear dengan *linked list*. Untuk program yang lebih besar, performa dapat ditingkatkan dengan menggunakan struktur data *hash table* atau *tree-based lookup* yang memberikan kompleksitas waktu $O(1)$ atau $O(\log n)$ untuk pencarian *identifier*.
3. **Validasi *Control Flow* yang Lebih Mendalam:** *Semantic analyzer* dapat diperluas untuk mendeteksi *unreachable code* (kode yang tidak akan pernah dieksekusi), *missing return statement* pada fungsi, atau *uninitialized variable usage* yang dapat menyebabkan *runtime error*.
4. **Penguatan Integrasi Antar Visitor:** Pengerjaan modular dengan berbagai visitor terbukti efektif. Namun, pendekatan ini sangat bergantung pada konsistensi penggunaan Symbol Table dan anotasi AST antar visitor. Disarankan untuk membuat *base visitor class* yang lebih kuat dengan *helper methods*

standar untuk operasi umum seperti *type inference*, *scope lookup*, dan *error reporting* agar kode lebih *maintainable* dan *extensible*.

5. **Peningkatan *Error Recovery***: Saat ini, semantic analyzer menghentikan proses ketika menemukan error pertama. Implementasi *error recovery* yang memungkinkan analyzer melanjutkan validasi dan melaporkan *multiple semantic errors* sekaligus akan sangat membantu *programmer* dalam *debugging* program yang kompleks.

LAMPIRAN

A. Github

<https://github.com/DitaMaheswari05/PSC-Tubes-IF2224.git>

Milestone 3 : <https://github.com/DitaMaheswari05/PSC-Tubes-IF2224/tree/feat/mils3>

B. Pembagian Tugas

(atur atur aja gais)

Nama / NIM	Symbol Table, DecoratedAST	Visitor	Integrasi	Laporan
Dita Maheswari / 13523125	100%	30%		20%
Boye Mangaratua Ginting / 13523127		30%		30%
Samantha Laqueenna Ginting / 13523138		30%		30%
Lukas Raja Agripa / 13523158		10%	100%	20%

NIM	Nama	Pekerjaan
13523125	Dita Maheswari	Symbol Table, DecoratedAST, Expression Visitor, Laporan
13523127	Boye Mangaratua Ginting	Statement Visitor, Laporan
13523138	Samantha Laqueenna Ginting	Declaration Visitor, Laporan
13523158	Lukas Raja Agripa	SemanticAnalyzer (main visitor), Integrasi, Laporan, Revisi DecoratedAST

REFERENSI

- <https://www.geeksforgeeks.org/compiler-design/phases-of-a-compiler/>
- https://www.tutorialspoint.com/compiler_design/compiler_design_attributed_grammars.htm
- <https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>